# CSCI 49900: Capstone

Group 8: Alex Haskel, James Lacoste, Kailun You

10/21/2026



# Overview

- Our project is an interactive platform to demonstrate the mechanics of large language model inference optimization and to allow for the exploration of model behavior through internal interventions.
- The website will allow: Prompt → baseline + configurable experimental run → side-by-side outputs + metrics.
- The project requires writing code to optimize a model, and creating an interactive platform in which to test model behavior and performance.
- Alex: Backend + interventions, James: Backend + inference optimization, Kailun: Frontend.



# Backend Development (Alex)

## Polished text generation limits:

- Text generation is capped by a user parameter: max_new_tokens
- Text is generated  until parameter reached or until end of sequence token
- **Issue:** Often, longer outputs were awkwardly truncated
- **Solution:** Use a custom stopping criterion
- Transformers library has a built in class StoppingCriteria
- Instead of producing max_new_tokens, we can use custom rules

# Backend Development (Alex)

```
#Absolute maximum amount of new tokens the model
#is allowed to generate.
hard_limit = int(settings.max_new_tokens*1.25)

#Custom stopping criteria to encourage stopping generation at
#more natural points in sentences, rather than mid-sentence.
criteria = SentenceBoundaryCriteria(
        tokenizer=tokenizer,
        prompt_length=inputs["input_ids"].shape[-1],
        soft_limit=settings.max_new_tokens
        )
```



Then, inside Boundary class:

```
if generated_ids.shape[1] < self.soft_limit:
    return torch.zeros(
```

```
for sequence in generated_ids:
    text = self.tokenizer.decode(sequence, skip_special_tokens=True)
    stop_conditions.append(text.rstrip().endswith(('.', '?', '!')))
```

# Expanded Backend Tests (Alex)

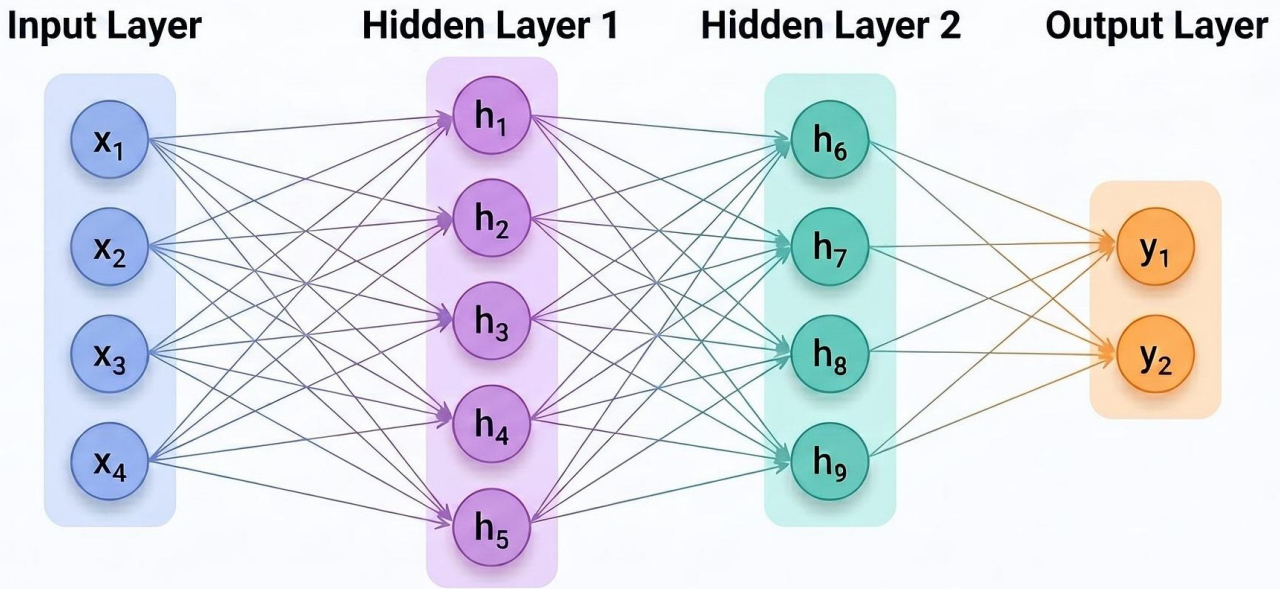


- Added tests for all generation functions, model interventions, and new stopping criteria behaviors.
- Used **MagicMock** and **monkeypatch** to create dummy objects and methods.
- E.g.: We may test aspects of text generation logic without having to load a model with millions of parameters every time.
- Using MagicMock, create a fake model, which is a dummy object, but it can respond to the same requests as the real one.

# Intervention Backend Tests (Alex)

- This is more complicated, because we have to simulate what would happen if we ablated some part of the model. We can't use any dummy object.

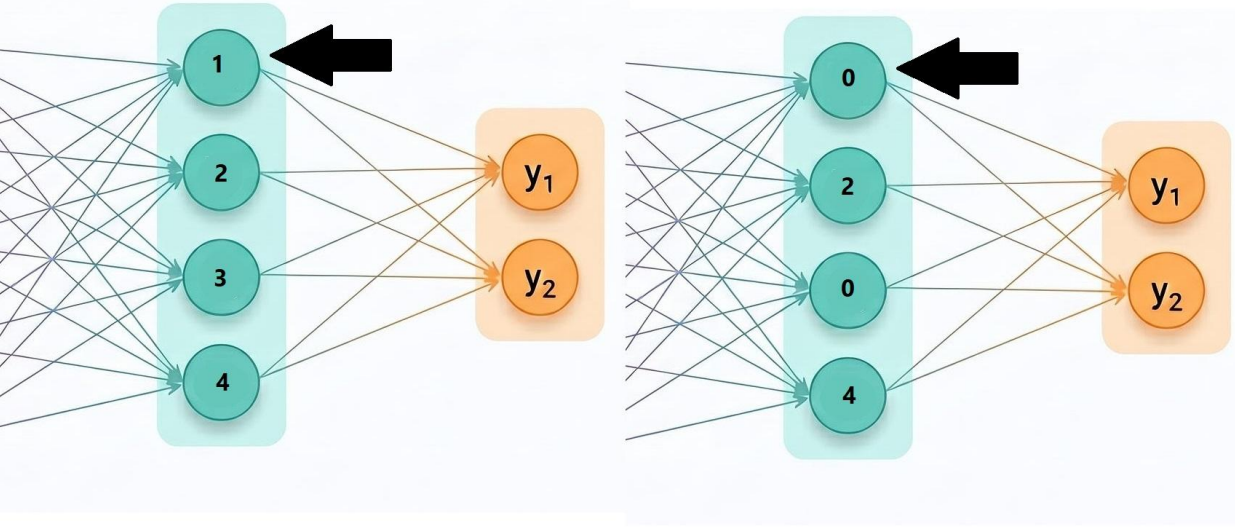
- **Solution:** Create a real PyTorch object that is a tiny version of the real 600 million parameter model.



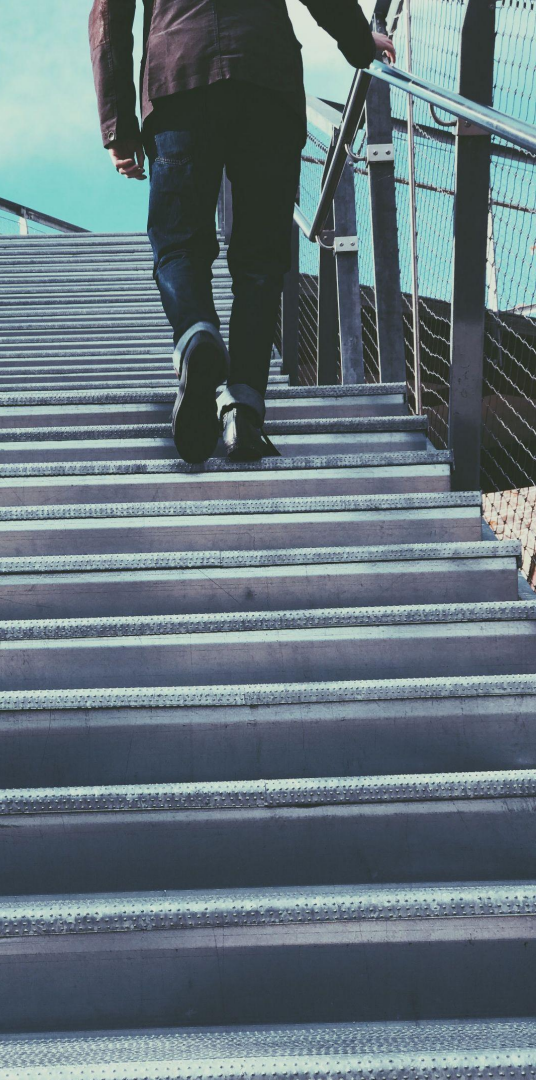

- Hidden layers in the real model have 3072 units. I created a tiny model with 4 units per layer.

# Intervention Backend Tests (Alex)

```python
def test_mlp_ablation():
    model = TinyModel()
    handle, indices, layer_idx, proportion = interventions.add_mlp_ablation(model=model,layer_idx=0,proportion=0.5)
    assert len(indices) == 2
    x = torch.tensor([[1.0, 2.0, 3.0, 4.0]])
    model.model.layers[0].mlp.down_proj(x)
    modified = model.model.layers[0].mlp.down_proj.last_input
    assert torch.all(modified[..., indices] == 0)
```




```python
class TinyModel(nn.Module):
    def __init__(self):
        super().__init__()
        self.model = TinyInnerModel()
        self.config = SimpleNamespace(
            num_hidden_layers=1,
            intermediate_size=4,
            num_attention_heads=4
        )
```



# Next Steps (Alex)

- Basic backend functions work and have now been tested both ‘manually’ through the API, and also through formal testing methods.
- Next step is to integrate back end and front end.
- Another important backend step would be to start measuring and returning real latency and throughput metrics for generated text. So far, I’ve been using placeholders.
- Also, need more sophisticated error handling.

# Kailun You

**Responsibility: Frontend**

## *Currently…*

01

Styling of the website (CSS)

02

Debug Option

# Kailun You

**Responsibility: Frontend**

## *Planning...*

01

More improvements on CSS and organize it

02

Complete Debug and allow website to create actual JSON files to send into backend.

03

Complete all the function of the website. (Single or Double option)

# Benchmarking Decode

For the Qwen3-0.6B model:

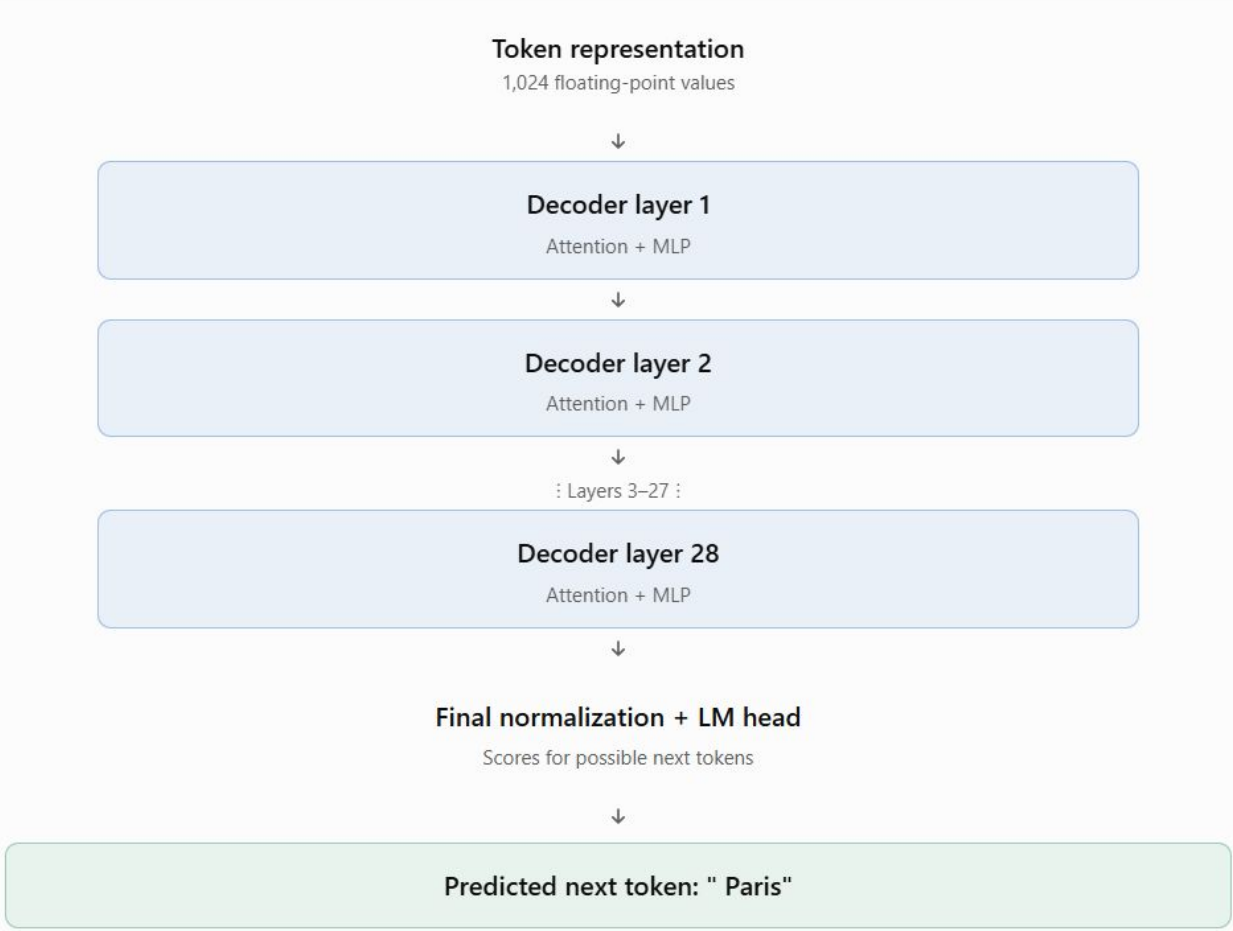
- One token is generated at a time
- For each token the model runs 28 identical decoder layers, and each layer multiplies the 1,024-value hidden vector by 7 weight matrices

Example next token generation:

"The capital of France is ….."

- The “hidden state” mathematically encodes the current text it is processing
- In our model, the hidden state is a vector of 1024 numbers
- The hidden state must pass through the entire model, however during this process a large amount of time is spend reading memory

# 28 Decode layers



# EACH layer has 7 matrices

## Attention: 4 matrices

Q, K, V, O

### q_proj

2,048 × 1,024

What information am I looking for?

### k_proj

1,024 × 1,024

What information can I match?

### v_proj

1,024 × 1,024

What information can I provide?

### o_proj

1,024 × 2,048

Combine attention outputs.

## MLP: 3 matrices

Gate, Up, Down

### gate_proj

3,072 × 1,024

Control features

### up_proj

3,072 × 1,024

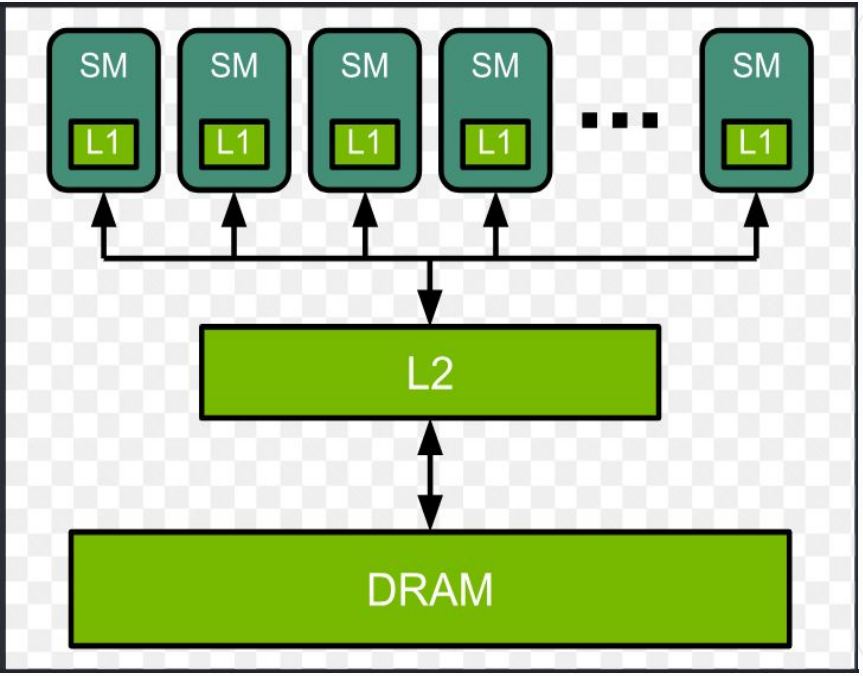
Expand features

### down_proj

1,024 × 3,072

Compress back

# The **important** part: The seven matrices are read from RAM




**Simplified GPU memory hierarchy**

**Registers / on-chip SRAM** — Fastest

Small storage very close to the compute units. Holds operands and intermediate results.

↑↓
Data movement

**GPU cache (L1 / L2)**

Larger than registers, but still limited. Can retain recently used weights and activations.

↑↓
Limited bandwidth

**GPU DRAM (HBM / GDDR)** — Largest

Stores the model weights, KV cache, and other tensors. Much greater capacity, but moving data to compute units takes time.

# The weight matrices are too large

- For examples, q_proj has 2,097,152 weights
- Assuming Floating point 16 datatype each weight is 2 bytes:

$$2{,}097{,}152 * 2 = 4{,}194{,}304$$

- That's about 4 MB

# Memory traffic for 7 matrices

| Matrix | Parameters | Weight size |
|---|---|---|
| `q_proj` | 2,097,152 | 4.19 MB |
| `k_proj` | 1,048,576 | 2.10 MB |
| `v_proj` | 1,048,576 | 2.10 MB |
| `o_proj` | 2,097,152 | 4.19 MB |
| `gate_proj` | 3,145,728 | 6.29 MB |
| `up_proj` | 3,145,728 | 6.29 MB |
| `down_proj` | 3,145,728 | 6.29 MB |
| **Total per layer** | **15,728,640** | **31.46 MB** |

# The weight matrices are too large

- 31.46 MB * 28 layers = 881 MB for decode
- The attention is another 311 MB (calculated similarly)
- 881 MB + 311 MB = 1192 MB or **1.19 GB of weight data per generated token**

- The GPU can perform the multiplications very fast, but spends a lot of time moving data
- My GPU of course can’t fit all the weight data for a token inside the L2 cache

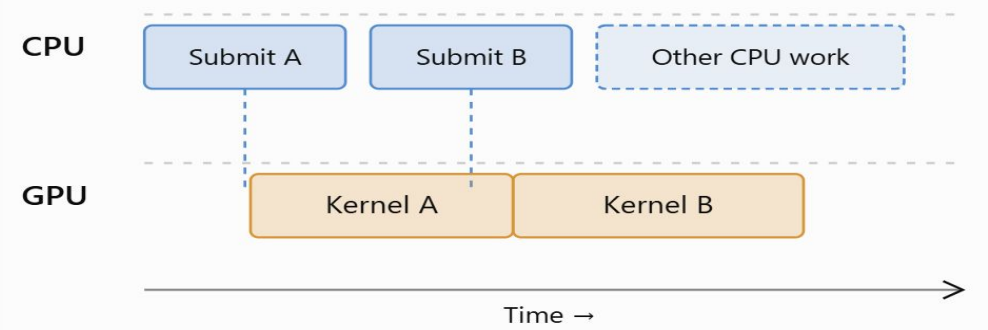
# How to evaluate/measure GPU work

- This naive way of measuring won't work!
- This only measures elapsed CPU time around the Python call
- We need to isolate the GPU instead

```
import time

start = time.perf_counter()
y = torch.matmul(W, x)
end = time.perf_counter()

print(end - start)
```




The CPU may submit kernel B before the GPU has finished kernel A. Operations in the same CUDA stream still execute in stream order.

```python
def time_cuda(
    fn: Callable[[], object],
    warmup: int = 10,
    iters: int = 50,
    flush: L2Flusher | None = None,
    queue_ahead_ms: float = 20.0,
) -> list[float]:
    for _ in range(warmup):
        if flush is not None:
            flush.flush()
        fn()
    starts = [torch.cuda.Event(enable_timing=True) for _ in range(iters)]
    ends = [torch.cuda.Event(enable_timing=True) for _ in range(iters)]
    if queue_ahead_ms > 0:
        torch.cuda._sleep(int(queue_ahead_ms * 1e-3 * 2e9))  # cycles; at least queue_ahead_ms below 2 GHz
    for start, end in zip(starts, ends):
        if flush is not None:
            flush.flush()
        start.record()
        fn()
        end.record()
    torch.cuda.synchronize()
    return [start.elapsed_time(end) for start, end in zip(starts, ends)]


def rotation_count(bytes_per_call: int, l2_bytes: int | None = None, factor: int = 2) -> int:
    if l2_bytes is None:
        l2_bytes = torch.cuda.get_device_properties(torch.cuda.current_device()).L2_cache_size
    return math.ceil(factor * l2_bytes / bytes_per_call) + 1
```

| Component | Purpose |
| --- | --- |
| `warmup` | Avoid including first-use effects such as lazy initialization, compilation, or initial clock ramp-up |
| `torch.cuda.Event` | Insert GPU-side timestamps |
| `_sleep()` | Give Python time to fill the GPU work queue |
| `flush.flush()` | Disturb or evict cached data to study a colder-memory condition |
| `start.record()` | Timestamp the beginning of the measured GPU interval |
| `fn()` | Submit the operation being benchmarked |
| `end.record()` | Timestamp the end of the measured interval |
| `torch.cuda.synchronize()` | Wait until submitted GPU operations have finished |
| `elapsed_time()` | Calculate elapsed milliseconds between event timestamps |

# One issue I was running into

- The timer has some uncertainty which is spread across many operations, so I record 50 operations and measure the average across all 50
- The first call might fetch the matrix from DRAM, but later calls might find the weights in L2 cache
- However, the real model does not use the exact same matrices every operation. Remember MLP has 3 matrices and attention has 4 matrices for each layer
- To solve this, I create rotated copies of the each matrix. This forces the GPU to read from RAM, which it would in the real model

```python
def time_cuda_rotating(
    fns: Sequence[Callable[[], object]], warmup: int = 3, iters: int = 50, queue_ahead_ms: float = 20.0
) -> list[float]:
    for _ in range(warmup):
        for fn in fns:
            fn()
    starts = [torch.cuda.Event(enable_timing=True) for _ in range(iters)]
    ends = [torch.cuda.Event(enable_timing=True) for _ in range(iters)]
    if queue_ahead_ms > 0:
        torch.cuda._sleep(int(queue_ahead_ms * 1e-3 * 2e9))
    for start, end in zip(starts, ends):
        start.record()
        for fn in fns:
            fn()
        end.record()
    torch.cuda.synchronize()
    return [start.elapsed_time(end) / len(fns) for start, end in zip(starts, ends)]
```

**Currently working on a more advanced statistics library to evaluate the performance based on our measured time**

```python
import statistics

def summarize(values) -> dict:
    v = sorted(values)
    if not v:
        raise ValueError("no values to summarize")
    q25, median, q75 = statistics.quantiles(v, n=4, method="inclusive") if len(v) > 1 else (v[0],) * 3
    return {
        "n": len(v),
        "median": median,
        "q25": q25,
        "q75": q75,
        "iqr": q75 - q25,
        "mean": statistics.fmean(v),
        "std": statistics.stdev(v) if len(v) > 1 else 0.0,
        "min": v[0],
        "max": v[-1],
    }

def bytes_gemv(n: int, k: int, weight_bytes_per_elem: float, act_bytes_per_elem: int, scale_bytes: int = 0) -> int:
    return int(n * k * weight_bytes_per_elem) + scale_bytes + (k + n) * act_bytes_per_elem

def gbs(nbytes: int, ms: float) -> float:
    return nbytes / (ms * 1e-3) / 1e9
```

# Next Steps

- Try to utilize the API endpoints that Alex made to connect with the quantized models, so we will be able to prompt both models through the API
- Since my teammates will be connecting the backend and frontend, it is a good opportunity to test and implement functionality to prompt both models through a visual textbox
- I will present more advanced numerical results on decode performance for my GPU
- Continue with kernel development